

Lab Work 3
Course Title: Information Security Laboratory/Project
Course Code: 0714 02 CSE 4108
Name: Sohana Rahman Tuly
Student ID: 230217

1. Implementation of DES (Single Round)

Source code:

```
s = [  
    14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7,  
    0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,  
    4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0,  
    15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13,  
    15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10,  
    3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,  
    0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15,  
    13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9,  
    10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8,  
    13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,  
    13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7,  
    1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12,  
    7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15,  
    13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,  
    10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4,  
    3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14,  
    2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9,  
    14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,  
    4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14,  
    11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3,
```

```

12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11,
10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,
9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6,
4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13,
4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1,
13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,
1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2,
6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12,
13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7,
1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,
7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8,
2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11
]
p = [
16,7,20,21,29,12,28,17,
1,15,23,26,5,18,31,10,
2,8,24,14,32,27,3,9,
19,13,30,6,22,11,4,25
]

```

```

def get_bit(value, index):
    return (value >> (31 - index)) & 1

def print_mat(desc, mat, bit_len, cols):
    print(f"\n{desc}:")
    for i in range(bit_len):
        bit = (mat >> (bit_len - 1 - i)) & 1

```

```

        print(bit, end="")

    if (i + 1) % cols == 0:
        print()
    else:
        print(" ", end="")

def cipher(l0, r0, k0):
    ex_r0 = 0
    for i in range(8):
        left = get_bit(r0, (i*4 + 31) % 32)
        mid = (r0 >> (28 - i*4)) & 15
        right = get_bit(r0, (i*4 + 4) % 32)
        ex_r0 |= ((left << 5) | (mid << 1) | right) << (42 - i*6)
    print_mat("Expanded r0", ex_r0, 48, 6)
    print_mat("K0", k0, 48, 6)
    r0_k0 = ex_r0 ^ k0
    print_mat("Expanded R0 ^ K0", r0_k0, 48, 6)
    s_out = 0
    for i in range(8):
        block = (r0_k0 >> (42 - i*6)) & 63
        row = ((block >> 4) & 2) | (block & 1)
        col = (block >> 1) & 15
        s_value = s[i*64 + row*16 + col]
        s_out |= s_value << (28 - i*4)
    print_mat("S box", s_out, 32, 4)
    p_out = 0
    for i in range(32):

```

```

        p_out |= get_bit(s_out, p[i] - 1) << (31 - i)
    print_mat("Permuted s box output", p_out, 32, 8)
    print_mat("L0", l0, 32, 8)
    l1 = r0
    r1 = p_out ^ l0
    c1 = (r1 << 32) | l1
    print_mat("R1 = permuted s box ^ L0 box", r1, 32, 8)
    print(f"\nL1: {l1:08X}")
    print(f"R1: {r1:08X}")
    print(f"C1: {c1:016X}")
    return c1

```

```

def main():
    text = input("Enter plaintext: ")
    key = input("Enter key: ")
    l0 = int(text[0:8], 16)
    r0 = int(text[8:16], 16)
    k0 = int(key, 16)
    print(f"L0: {l0:08X}")
    print(f"R0: {r0:08X}")
    cipher(l0, r0, k0)

```

```

if __name__ == "__main__":
    main()

```

Input:

Enter plaintext: 123456ABCD132536

Enter key: 1B02EFFC7072

Output:

L0: 123456AB

R0: CD132536

Expanded r0:

0 1 1 0 0 1

0 1 1 0 1 0

1 0 0 0 1 0

1 0 0 1 1 0

1 0 0 1 0 0

0 0 1 0 1 0

1 0 0 1 1 0

1 0 1 1 0 1

K0:

0 0 0 1 1 0

1 1 0 0 0 0

0 0 1 0 1 1

1 0 1 1 1 1

1 1 1 1 1 1

0 0 0 1 1 1

0 0 0 0 0 1

1 1 0 0 1 0

Expanded $R0 \wedge K0$:

0 1 1 1 1 1

1 0 1 0 1 0

1 0 1 0 0 1

0 0 1 0 0 1

0 1 1 0 1 1

0 0 1 1 0 1

1 0 0 1 1 1

0 1 1 1 1 1

S box:

1 0 0 0

0 1 0 0

0 1 1 0

0 1 1 0

1 0 0 1

1 0 0 1

1 0 0 0

0 0 1 0

Permuted s box output:

0 0 1 1 0 0 0 1

1 1 0 0 0 0 1 1

0 0 1 1 0 0 0 0

0 0 0 1 0 1 0 1

L0:

0 0 0 1 0 0 1 0

0 0 1 1 0 1 0 0

0 1 0 1 0 1 1 0

1 0 1 0 1 0 1 1

R1 = permuted s box ^ L0 box:

0 0 1 0 0 0 1 1

1 1 1 1 0 1 1 1

0 1 1 0 0 1 1 0

1 0 1 1 1 1 1 0

L1: CD132536

R1: 23F766BE

C1: 23F766BECD132536